PYTHON 조건문

PYTHON 조건문

◆ 조건문

- 프로그램 실행이 **특정 조건에 따라 실행 방향 변경**
- 조건이 **참인 경우와 거짓인 경우**에 따라서 실행

- 주의 !!!
  - 조건문에 속하는 모든 문장은 **들여쓰기**

PYTHON 조건문

◆ 조건문

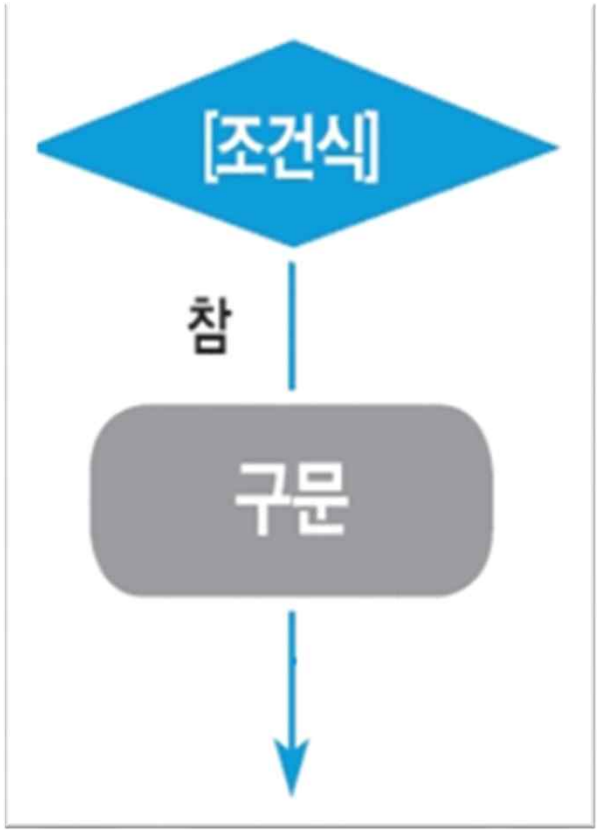
- 자료형에 따른 조건 결과

자료형	참	거짓
숫자형 타입	0이 아닌 모든 숫자	0
문자형 타입	"abc"	""
리스트 타입	[1,2,3,4,5]	[ ]
튜 플 타입	(1, 2, 3, 4, 5)	( )
딕셔너리 타입	{ "A" : 90 }	{ }

PYTHON 조건문

◆ 조건문

➢ 단순 조건문

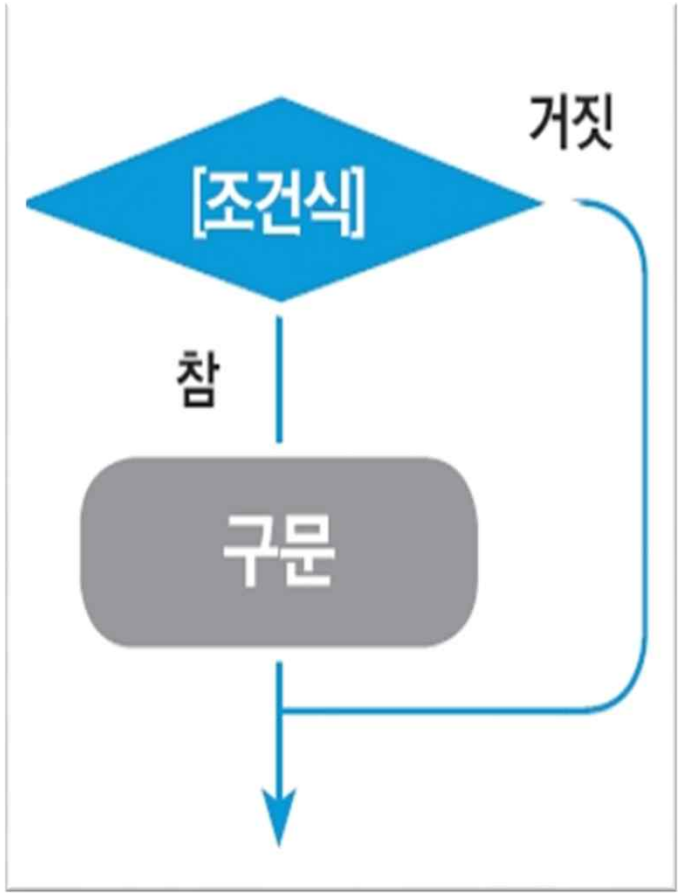



```
if 조건식:
    구문
    구문
    구문
```

PYTHON 조건문

◆ 조건문

➢ 단순 조건문




```
if 조건식:
    구문
    구문
else :
    구문
    구문
```

PYTHON 조건문

◆ 조건문

➢ 예제

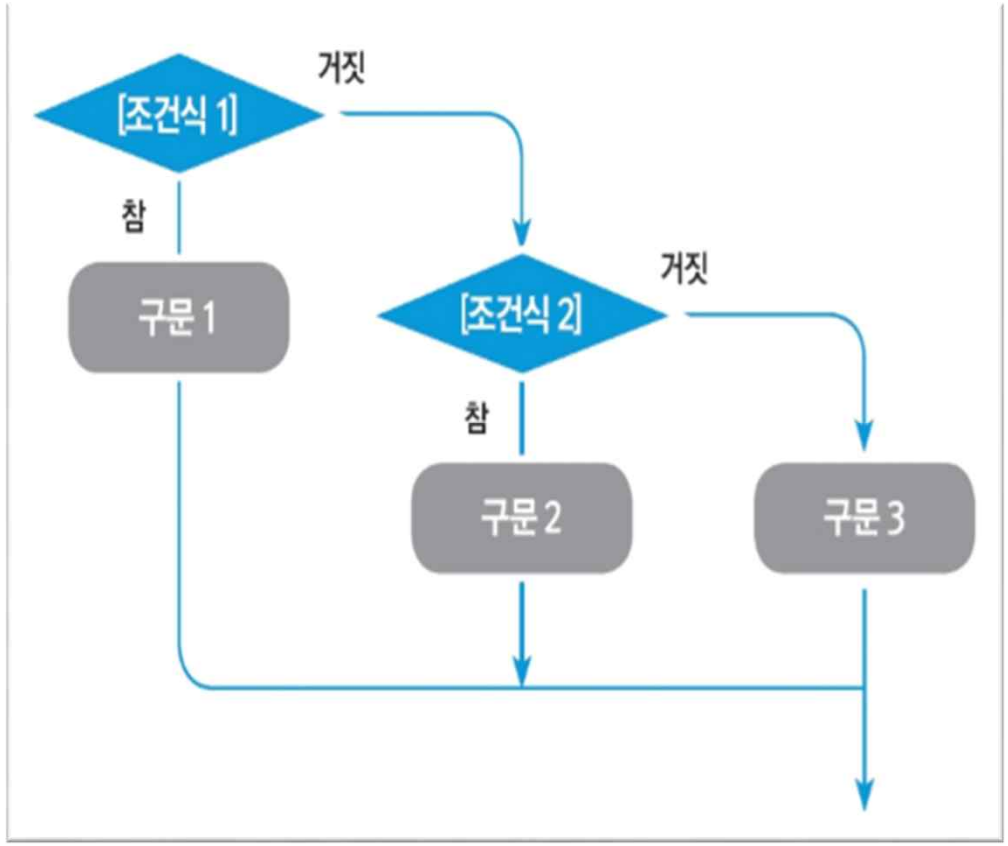
```
if score >= 60 :
    print("합격입니다.")
else:
    print("불합격입니다.")
```

```
if num % 2 == 0 :
    print("짝수입니다.")
else:
    print("홀수입니다.")
```

PYTHON 조건문

◆ 조건문

➢ 다중 조건문




```
if 조건식1:
    구문
    구문
elif 조건식2:
    구문
    구문
else:
    구문
```

PYTHON 조건문

◆ 조건문

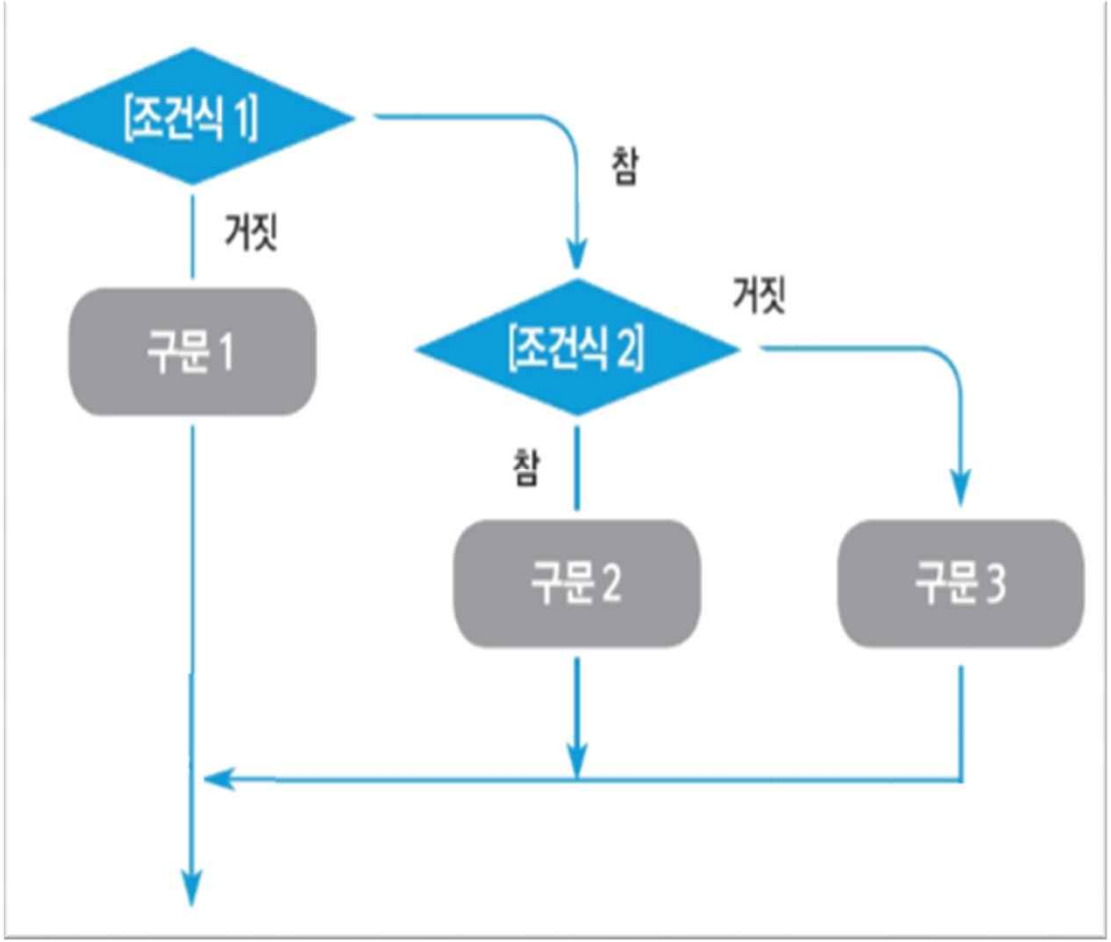
➢ 다중 조건문

```
if value>0:
    print("양수")
elif value <0:
    print("음수")
else:
    print("0")
```

PYTHON 조건문

◆ 조건문

➢ 중첩 조건문




```
if  조건식1:
        if 조건식2:
            구문2-1
        else:
            구문2-2
else:
        구문1
```

PYTHON 조건문

◆ 조건문

➢ 중첩 조건문

```
if value>0:
    if value%2:
        print( " 홀수")
    else:
        print("짝수")
else:
    print( " 음수")
```

PYTHON 조건문

◆ 조건문

멤버 연산자와 조건문

in	not in
X in 리스트	X not in 리스트
X in 튜플	X not in 튜플
X in 문자열	X not in 문자열

```
>>> 1 in [1, 2, 3]          True
>>> 1 not in [1, 2, 3]      False
```

```
>>> 'a' in ['a', 'b', 'c']      True
>>> 'i' not in 'python'         True
```

PYTHON 조건문

◆ 조건문

조건부 표현식

참인 경우 if 조건문 else 거짓인 경우

```
print("odd number") if value%2 else print("even number")

kind="Odd Number" if value%2 else "Even Number"
```

PYTHON 조건문

◆ 조건문

조건부 표현식

```
print("홀수") if value%2 else print("짝수")

print("짝수") if value%2==0 else print("홀수")

msg="홀수" if value%2 else "짝수"

msg="짝수" if value%2==0 else "홀수"
```

PYTHON 반복문

PYTHON 반복문

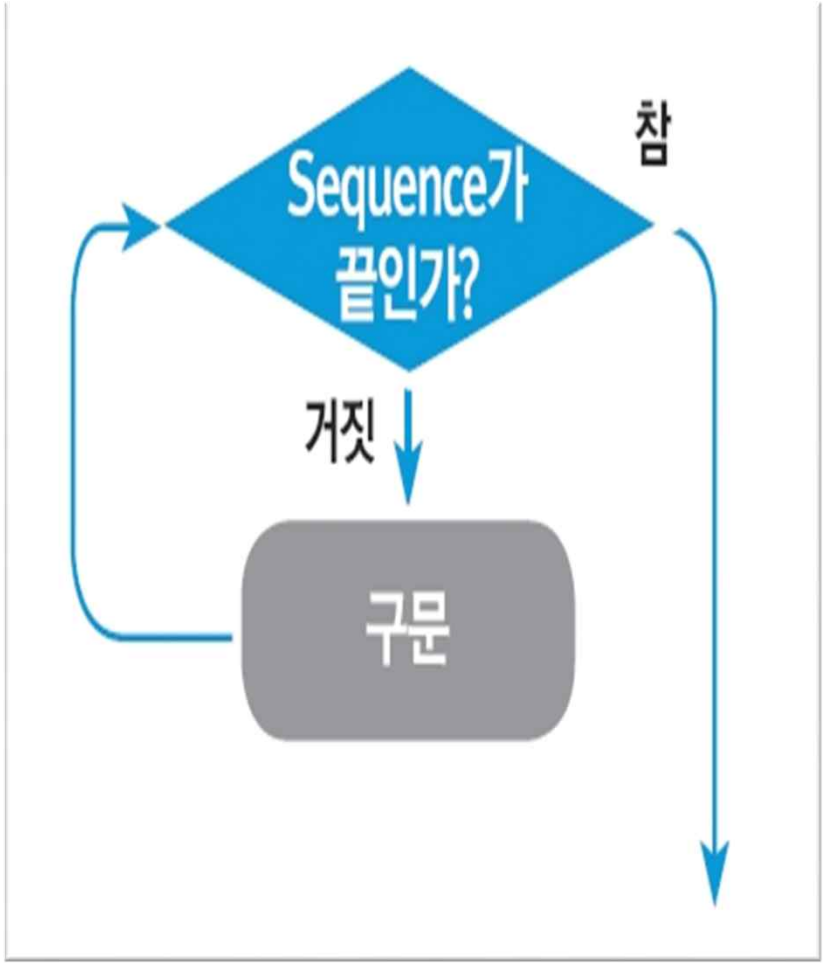
◆ 반복문

- **동일한 코드 반복**해야 할 경우 사용되는 제어문

- 종류 : for문, while문

PYTHON 반복문

◆ 반복문 - for



for 요소 **in** 시퀀스 객체 **:**

반복문

시퀀스객체에서

하나씩 원소(요소) 가져오기

PYTHON 반복문

◆ 반복문 - for

```
data=[10, 3.25, "A"]
for item in data:
    print( item )
```

```
data=[11,22,33,44,55]
for item in data:
    print( type(item), item )
```

```
data="Good Luck"
for item in data:
    print( item, end='₩t' )
```

PYTHON 반복문

◆ 반복문 - for

```
data={'name':'Hong', 'age':10, 3:10.23}

for (k, v) in data.items():
    print("Item => {},{}".format(k, v))
```

```
a=[(1,2), (3,4),(5,6)]

for (first, last) in a:
    print( "{0} + {1} = {2}".format(first, last, first + last) )
```

PYTHON 반복문

◆ 반복문 - for

```
for 요소 in range(종료 값) :
    반복문
```

```
for 요소 in range(시작, 종료, 증가) :
    반복문
```

PYTHON 반복문

◆ 반복문 - for

```
for i in range(10):
    print("Item => {}".format(i), end='₩t')
print("")
```

```
for i in range(2, 11, 2) :
    print("Item => {}".format(i), end='₩t')
print("")
```

PYTHON 반복문

◆ 반복문 - for

중첩 for

```
for 요소 in 시퀀스 객체 :
    for 요소 in 시퀀스 객체 :
        반복문
```

PYTHON 반복문

◆ 반복문 - for

중첩 for

```
for dan in range(2,10):
    print(f"======{dan}단=====")

    for num in range(1,10):
        print(f"{dan}*{num}={dan*num}")
```

PYTHON 반복문

◆ 반복문 - for

- Comperhesion
  - 반복 표현식을 포함하여 변형된 다른 Sequence 생성

```
list_objet = [ 표현식 for 요소 in 반복가능 객체  if 조건 ]
             [ 표현식 if 조건 else  표현식 for 요소 in 반복가능 객체 ]
```

```
set_objet = { 표현식 for 요소 in 반복가능 객체  if 조건 }
            { 표현식 if 조건 else  표현식 for 요소 in 반복가능 객체 }
```

```
dict_objet = { key:value for 요소 in 반복가능 객체  if 조건 }
             { key:value if 조건 else  표현식 for 요소 in 반복가능 객체 }
```

PYTHON 반복문

◆ 반복문 - for

```
aa=[1,2,3]
result=[]
for num in aa:
    result.append(num*3)
```

List Comprehension

```
result=[ num*3 for num in aa ]
```

```
result=[ num*3 for num in aa if num==2 ]
```

```
result=[ num*3 if num==2 else num for num in aa ]
```

PYTHON 반복문

◆ 반복문 - enumerate 객체

- 시퀀스 데이터의 요소 꺼낼때 **인덱스 부여 반환** 객체
- 반환 : enumerate 객체
- 문법 : range(data) : 인덱스 0부터 데이터 구성
  range(data,start=number) : 인덱스 number부터 데이터 구성

PYTHON 반복문

◆ 반복문 - iterable 객체

- **반복이 가능한 객체 의미**

- 타입 : 순서형 및 컬렉션 데이터 타입

  str, list, tuple, dict, set, bytes, range

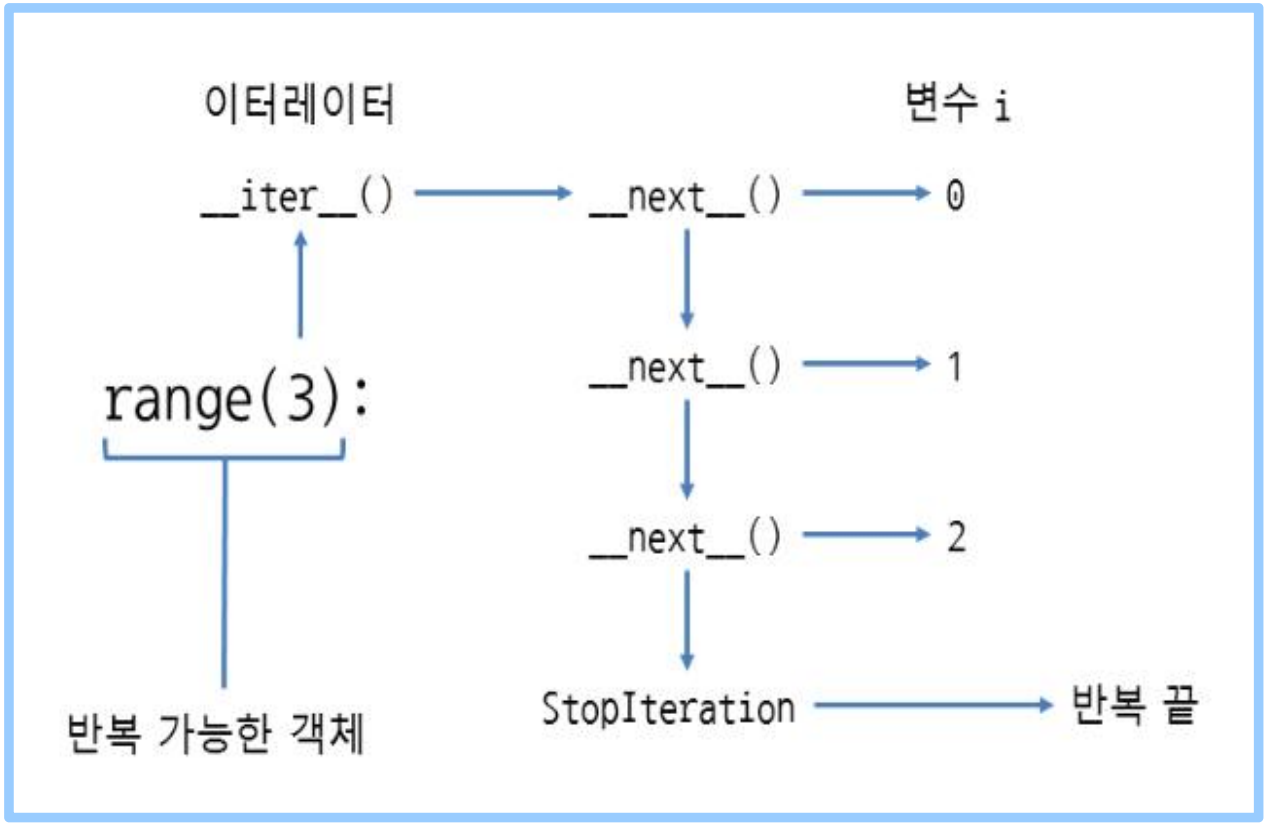
- 사용 : for ~ in 반복문

- 특징 : iterator 객체 생성 메서드 __iter__() 존재

PYTHON 반복문

◆ 반복문 - iterable 객체

- 동작원리



◆ 반복문 - Iterator 객체

- 데이터를 **차례대로 하나씩 꺼낼 수 있는 객체**
- **iterable한 객체로 생성된 객체**
- **반복자 패턴**이라고 함
- 한 번 데이터 사용 후 폐기

- 구성
  - 반복을 위한 첫 번째 요소 가리키는 메서드
  - 다음 요소 가리키는 메서드
  - 마지막 요소 가리키느 메서드
  - 현재 요소 반환하는 메서드

PYTHON 반복문

◆ 반복문 - Iterator 객체

- 생성 메서드 ➔ **iter(iterable)**
- 데이터 꺼내는 메서드 ➔ **next(iterator)**

```
numList = [1, 2, 3]
numListiter = iter(numList)

print(type(numListiter), numListiter)

try:
        # 내장함수 사용
        print(next(numListiter))
        print(next(numListiter))
        print(next(numListiter))
        print(next(numListiter))
except StopIteration:
    print("End of Iterator")
finally:
    print("END")
```

PYTHON 반복문

◆ 반복문 - Iterator 객체

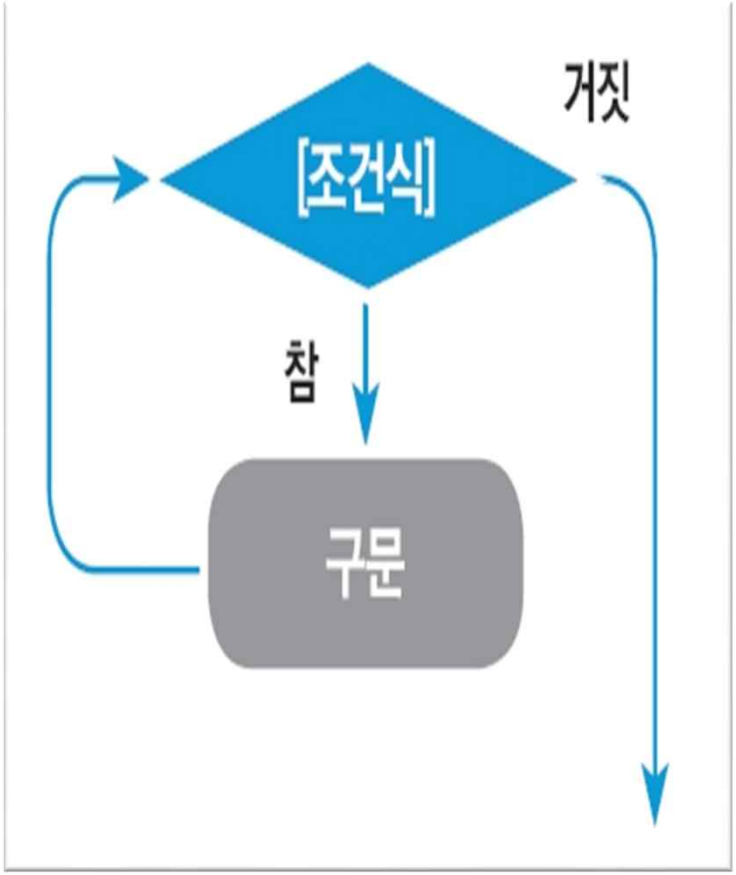
- 생성 메서드 ➔ **seqobj.__ iter__( )**
- 데이터 꺼내는 메서드 ➔ **seqobj.__next__( )**

```
numList = [1, 2, 3]
numListiter = iter(numList)

print(type(numListiter), numListiter)

try:
        # 메서드 사용
        print(numListiter.__next__())
        print(numListiter.__next__())
        print(numListiter.__next__())
        print(numListiter.__next__())
except StopIteration:
    print("End of Iterator")
finally:
    print("END")
```

PYTHON 반복문

◆ 반복문 - while

- 반복횟수가 명확하지 않는 경우 사용



초기값

while 조건식 :

실행문

초기값 갱신

반복 중단 위해 필수!

PYTHON 반복문

◆ 반복문 - while

```
num=20;                          # 초기값

while num>0:
    print(f"num is {num}")       # 반복
    num=num-1                    # 초기값 갱신
```

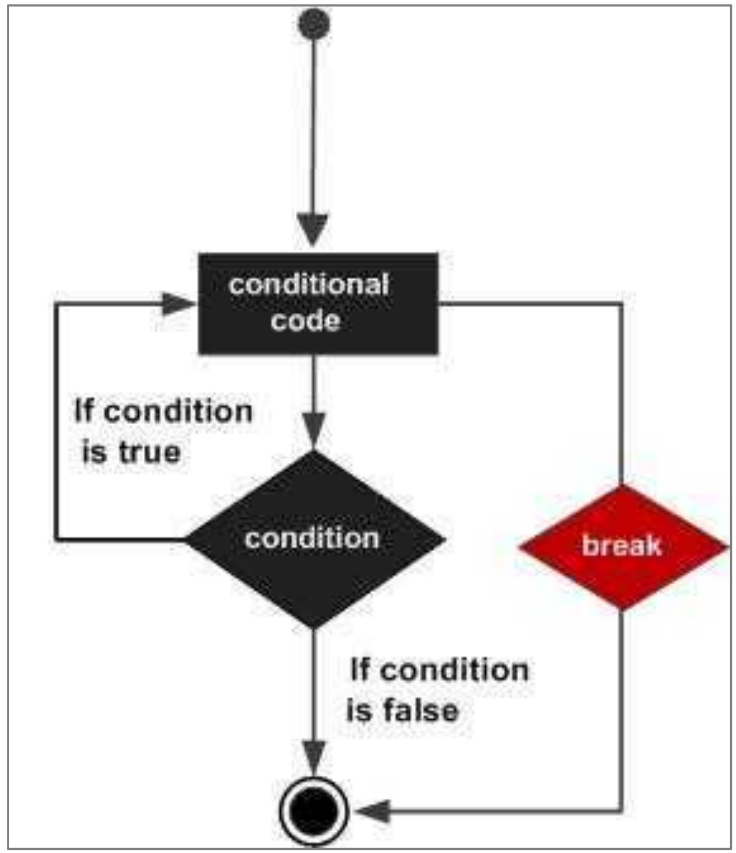
```
sum=0                            # 초기값

while sum<50:
    print(f"num is {sum}")
    sum=sum+5                    # 초기값 갱신
```

PYTHON 반복문

◆ 반복 중단

➢ break 문

- for와 while에서 반복 중단을 위해 사용



PYTHON 반복문

◆ 반복 중단

➢ break 문

```
for item in range(50):
    if item > 17:
        break
    print(f"Item : {item}")
```

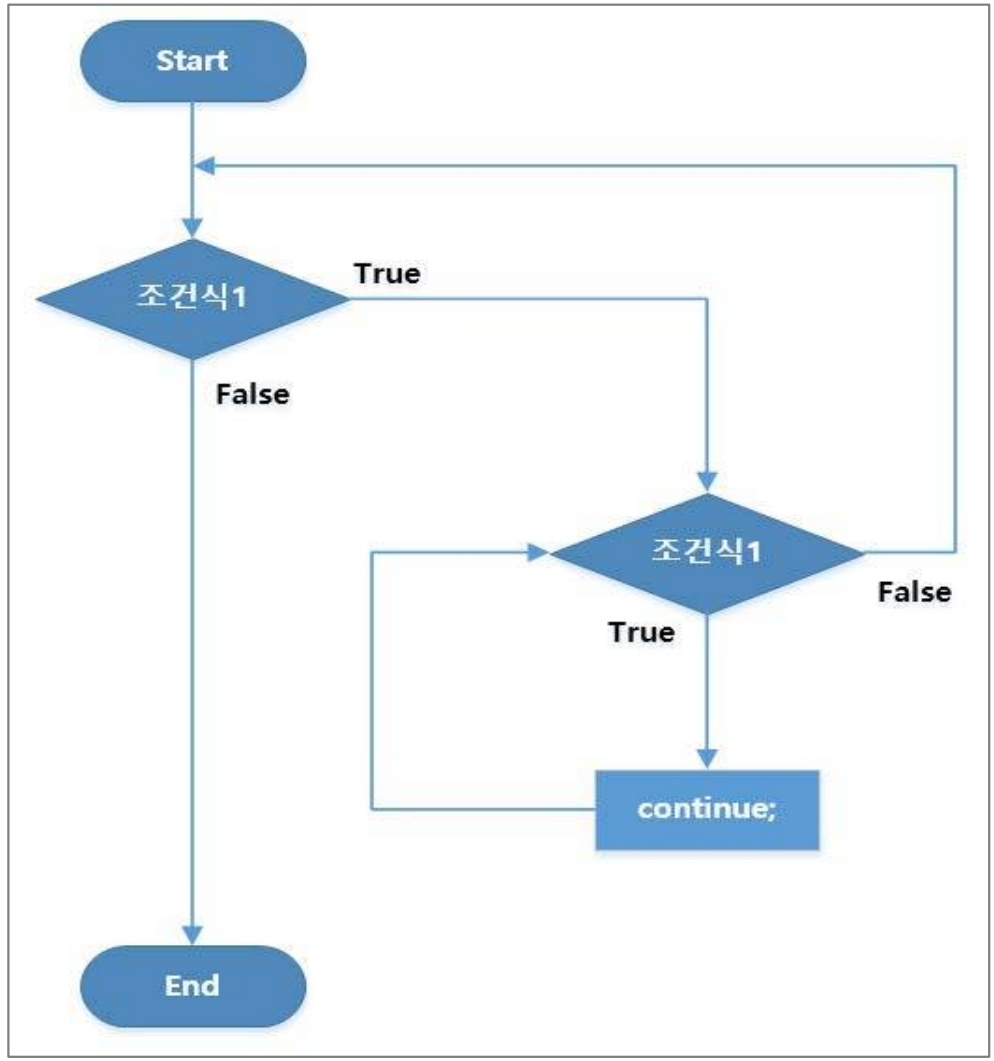
```
num=sum=0
print(num, sum)
while True:
    if sum >=33:
        print(" ==== ")
        break
    sum = sum + num
    num = num + 1
print(" END WHILE")
```

PYTHON 반복문

◆ 반복 스킵

➢ continue 문

- 반복문에서 일부코드
- 실행하지 않고
- 다음으로 진행



PYTHON 반복문

◆ 반복 스킵

➢ continue 문

```
for value in range(50):

    if  value%2==0:
        continue

    print(f"value : {value}")
```

PYTHON 반복문

◆ 조건 무시

➢ pass 문

```
i = 0
while i < 10:
    i += 1

    if  i % 2 ==0:
        continue

    print(i)
```

```
i = 0
while i < 10:
    i += 1

    if  i % 2 ==0:
        pass

    print(i)
```